

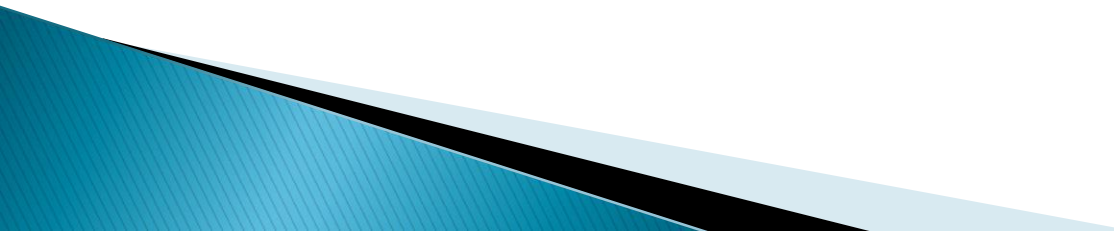
Despre algoritmi



De ce despre algoritmi?

- ▶ numeroase aplicații
- ▶ în practică este importantă eficiența algoritmilor
- ▶ ar fi util să știm dacă algoritmii pe care îi propunem sunt corecți
 - 😊 corectitudine $\neq$ nu a găsit cineva încă un contraexemplu

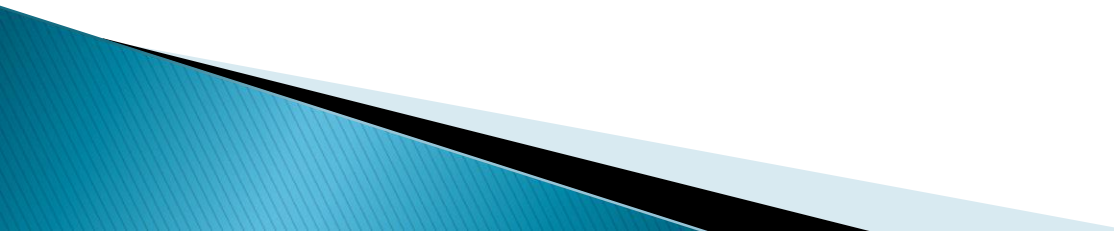
Aspecte generale care apar la rezolvarea unei probleme

- ▶ ***Teoretic***, pașii elaborării un algoritm sunt următorii:
 1. demonstrarea faptului că este **posibilă** elaborarea unui algoritm pentru determinarea unei soluții
 - 2.
 - 3.
 - 4.
 - 5.
- 

Aspecte generale care apar la rezolvarea unei probleme

- ▶ ***Teoretic***, pașii elaborării un algoritm sunt următorii:
 1. demonstrarea faptului că este **posibilă** elaborarea unui algoritm pentru determinarea unei soluții
 2. **elaborarea** algoritmului
 3. demonstrarea **corectitudinii** algoritmului
 - 4.
 - 5.

Aspecte generale care apar la rezolvarea unei probleme

- ▶ ***Teoretic***, pașii elaborării un algoritm sunt următorii:
 1. demonstrarea faptului că este **posibilă** elaborarea unui algoritm pentru determinarea unei soluții
 2. **elaborarea** algoritmului
 3. demonstrarea **corectitudinii** algoritmului
 4. determinarea **timpului de executare** a algoritmului
 5. demonstrarea **optimalității** algoritmului
- 

Existența algoritmilor



Existența algoritmilor

- ▶ **Problemă nedecidabilă** = pentru care nu poate fi elaborat un algoritm.
- 1. **Problema opririi programelor:** pentru orice program și orice valori de intrare să se decidă dacă programul se termină.
- 2. **Problema echivalenței programelor:** să se decidă pentru orice două programe dacă sunt echivalente (produc aceeași ieșire pentru aceleași date de intrare).

Elaborarea algoritmilor



Elaborarea algoritmilor

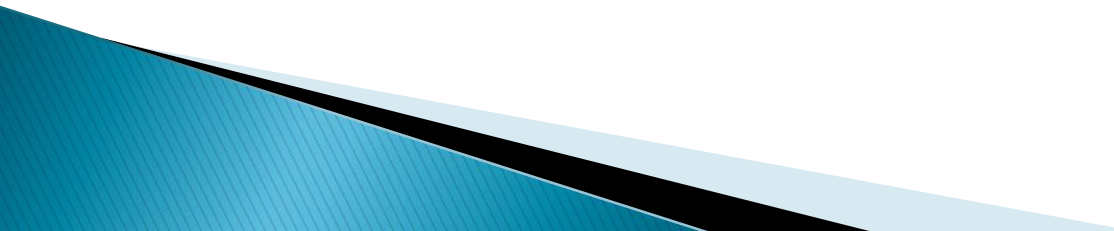
- ▶ **Cursurile următoare:** metode de elaborare a algoritmilor

Timpul de executare

Timpul de executare a algoritmilor

- ▶ se măsoară în funcție de lungimea n a datelor de intrare
- ▶ $T(n)$ = timpul de executare pentru orice set de date de intrare de lungime n
 - dat de numărul de operații elementare în funcție de n

Timpul de executare a algoritmilor

- ▶ Se numără operații elementare (de atribuire, aritmetice, de decizie, de citire/scriere)
 - ▶ Numărare aproximativă => **ordinul de mărime** al numărului de operații elementare
 - ▶ Pentru simplitate – se fixează operație de bază
- 

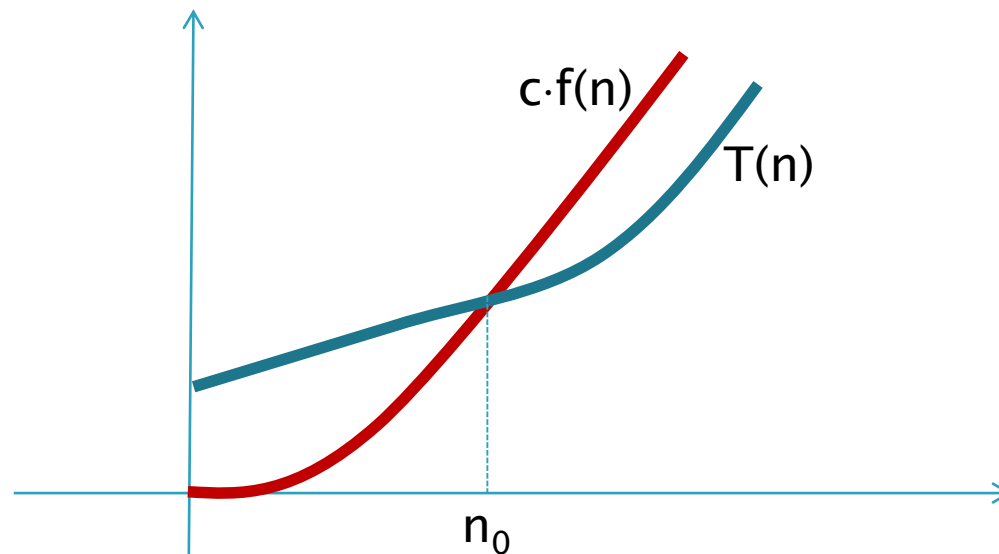
Timpul de executare a algoritmilor

- ▶ În majoritatea cazurilor ne mărginim la a evalua **ordinul de mărime** al timpului de executare = **ordin de complexitate** al algoritmului

$$T(n) = O(f(n))$$

$$\exists c, n_0 - \text{constante a.î } \forall n \geq n_0$$

$$T(n) \leq c \cdot f(n)$$

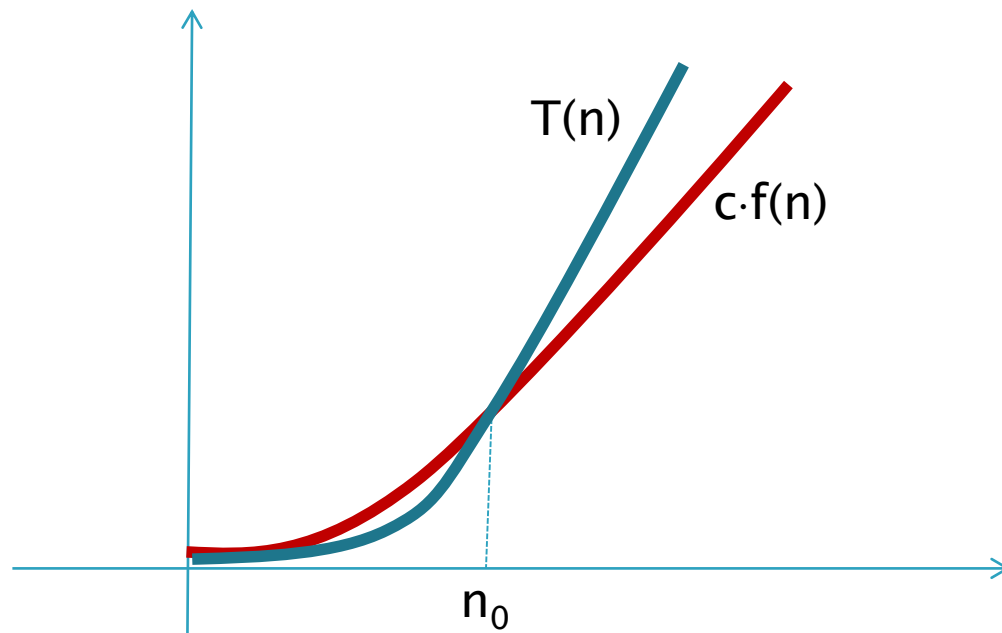


Timpul de executare a algoritmilor

- $T(n) = \Omega(f(n))$ (Suplimentar)

$\exists c, n_0$ - constante a.î $\forall n \geq n_0$

$$T(n) \geq c \cdot f(n)$$

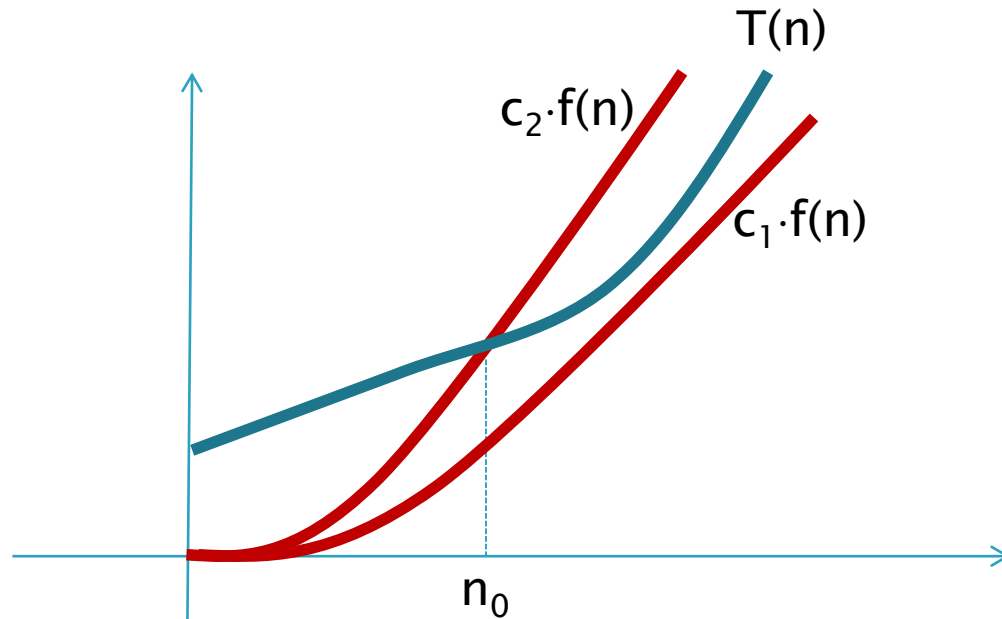


Timpul de executare a algoritmilor

- $T(n) = \Theta(f(n))$ (Suplimentar)

$\exists c_1, c_2, n_0$ constante a.î $\forall n \geq n_0$

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$$



Timpul de executare a algoritmilor

- ▶ Notatie: $T(n) = O(f(n))$
- ▶ comportare **asimptotică**
- ▶ caz defavorabil
- ▶ $O(\text{expresie}) = O(\text{termen dominant})$

$$O(2n) \Rightarrow O(n)$$

$$O(2n^2 + 4n + 1) \Rightarrow O(n^2)$$

$$O(n^2 - n) \Rightarrow O(n^2)$$

Timpul de executare a algoritmilor

▶ Exemplul 1

pentru $i = 1, n$ executa

 pentru $j = 1, i$ executa

 scrie j

 scrie linie noua

Timpul de executare a algoritmilor

► Exemplul 1

pentru $i = 1, n$ executa

 pentru $j = 1, i$ executa

 scrie j

 scrie linie noua

Afişare

1

1 2

1 2 3

1 2 3 4

1 2n

Timpul de executare a algoritmilor

► Exemplul 1

pentru $i = 1, n$ executa

 pentru $j = 1, i$ executa

 scrie j

 scrie linie noua

Afişare

1

1 2

1 2 3

1 2 3 4

1 2n

$1 + 2 + 3 + \dots + n = n(n+1)/2$ afişări ale lui j

$O(n^2)$

Timpul de executare a algoritmilor

► Exemplul 1

pentru $i = 1, n$ executa

 pentru $j = 1, i$ executa

 scrie j

 scrie linie noua

Afişare

1

1 2

1 2 3

1 2 3 4

1 2n

$1 + 2 + 3 + \dots + n = n(n+1)/2$ afişări ale lui j

$O(n^2)$

Timpul de executare a algoritmilor

► Exemplul 2

`p = 1`

`pentru i = 1, n+1 executa`

`pentru j = 1, p executa`

`scrie j`

`scrie linie noua`

`p = p * 2`

Timpul de executare a algoritmilor

► Exemplul 2

```
p = 1
```

```
pentru i = 1, n+1 executa
```

```
    pentru j = 1,p executa
```

```
        scrie j
```

```
    scrie linie noua
```

```
p = p * 2
```

Afişare

1

1 2

1 2 3 4

1 2 3 4 5 6 7 8

1 2 2^n

Timpul de executare a algoritmilor

► Exemplul 2

`p = 1`

`pentru i = 1, n+1 executa`

`pentru j = 1, p executa`

`scrie j`

`scrie linie noua`

`p = p * 2`

Afişare

1

1 2

1 2 3 4

1 2 3 4 5 6 7 8

1 2 2^n

$$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1 \text{ afişări ale lui } j$$

$O(2^n)$

Timpul de executare a algoritmilor

- ▶ **Exemplul 3 – Cea mai mică putere a lui 2 mai mare ca n**

```
p = 1
```

```
cat timp p<=n executa:
```

```
    p = p * 2
```

```
scrie p
```

Timpul de executare a algoritmilor

- ▶ Exemplul 3 – Cea mai mică putere a lui 2 mai mare ca n

```
p = 1
```

```
cat timp p<=n executa:
```

```
    p = p * 2
```

```
scrie p
```

$O(\log_2(n))$

Timpul de executare a algoritmilor

▶ Exemplul 4 – 2-SUM pentru șir crescător

Se dă un vector ordonat crescător cu n elemente întregi distincte.

Să se afișeze toate perechile de elemente din vector cu suma 0

Timpul de executare a algoritmilor

► Exemplul 4 – 2-SUM pentru șir crescător

pentru $i = 1, n-1$ executa

 pentru $j = i+1, n$ executa

 daca $v[i]+v[j]==0$ atunci

 scrie i, j

Timpul de executare a algoritmilor

► Exemplul 4 – 2-SUM pentru șir crescător

pentru $i = 1, n-1$ executa

 pentru $j = i+1, n$ executa

 daca $v[i]+v[j]==0$ atunci

 scrie i, j

$O(n^2)$

Timpul de executare a algoritmilor

► Exemplul 4 – 2-SUM pentru şir crescător

```
i = 0
```

```
j = n-1
```

```
cat timp i < j executa
```

```
    daca v[i]+v[j]==0 atunci
```

```
        scrie i,j
```

```
        i = i + 1
```

```
        j = j - 1
```

```
    altfel
```

```
        daca v[i] + v[j] < 0 atunci
```

```
            i = i + 1
```

```
        altfel
```

```
            j = j - 1
```

Timpul de executare a algoritmilor

► Exemplul 4 – 2-SUM pentru şir crescător

```
i = 0
```

```
j = n-1
```

```
cat timp i < j executa
```

```
    daca v[i]+v[j]==0 atunci
```

```
        scrie i,j
```

```
        i = i + 1
```

```
        j = j - 1
```

```
    altfel
```

```
        daca v[i] + v[j] < 0 atunci
```

```
            i = i + 1
```

```
        altfel
```

```
            j = j - 1
```

$O(n)$

Timpul de executare a algoritmilor

Alte exemple

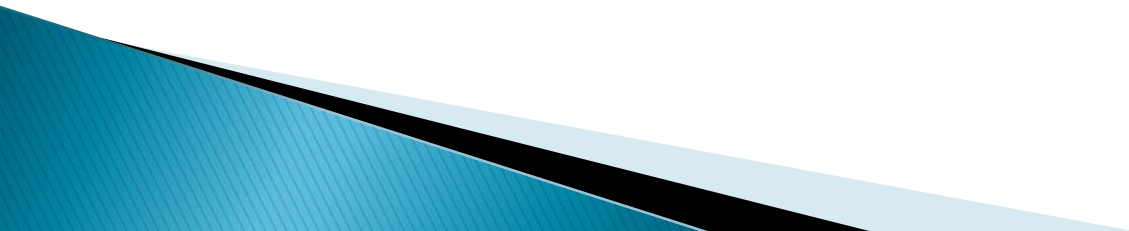
pentru $i = 1, n$ executa

pentru $j = 1, m$ executa

operatii $O(1)$

pentru $k = 1, p$ executa

operatii $O(1)$



Timpul de executare a algoritmilor

Alte exemple

pentru $i = 1, n$ executa

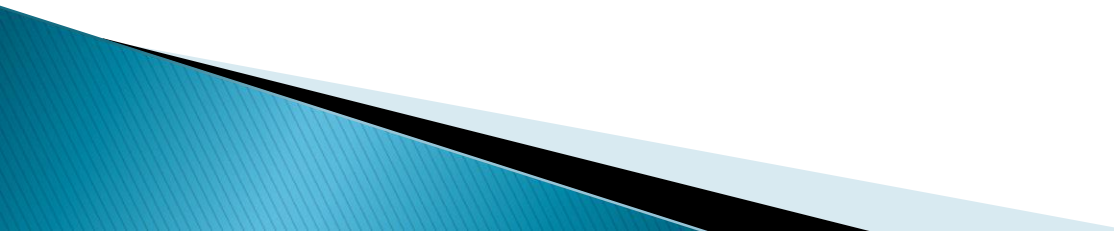
pentru $j = 1, m$ executa

operatii $O(1)$

pentru $k = 1, p$ executa

operatii $O(1)$

$O(n(m+p))$



Timpul de executare a algoritmilor

Alte exemple

- ▶ Înmulțirea a două matrice $A(n,m)$ $B(m,p)$
- ▶ Intersecția a două mulțimi cu n respectiv m elemente
- ▶ Reuniunea a două mulțimi **ordonate** cu n respectiv m elemente

Timpul de executare a algoritmilor

Alte exemple

- ▶ Înmulțirea a două matrice $A(n,m)$ $B(m,p)$ $O(nmp)$
- ▶ Intersecția a două mulțimi cu n respectiv m elemente $O(nm)$
- ▶ Reuniunea a două mulțimi **ordonate** cu n respectiv m elemente – similar Interclasare $O(n+m)$

Timpul de executare a algoritmilor

► Interclasare

```
i = 1; j = 1
```

```
cat timp (i<=m) and (j<=n) executa
```

```
    daca a[i]<=b[j] atunci
```

```
        scrie a[i]; i = i+1
```

```
    altfel
```

```
        scrie b[j]; j = j+1
```

```
cat timp i<=m executa
```

```
    scrie a[i]; i=i+1
```

```
cat timp j<=n executa
```

```
    scrie b[j]; j=j+1
```

Timpul de executare a algoritmilor

Notatie: $T(n) = O(f(n))$

Clase de complexitate uzuale:

- ▶ Complexitate logaritmică $O(\log_2(n))$, $O(\log(n))$

Exemplu: căutarea binară



Timpul de executare a algoritmilor

Notatie: $T(n) = O(f(n))$

Clase de complexitate uzuale:

- ▶ Complexitate logaritmică $O(\log_2(n))$, $O(\log(n))$

Exemplu: căutarea binară

- ▶ Complexitate liniară $O(n)$

Exemplu: minimul dintr-un vector

- ▶

Timpul de executare a algoritmilor

Notatie: $T(n) = O(f(n))$

Clase de complexitate uzuale:

- ▶ Complexitate logaritmică $O(\log_2(n))$, $O(\log(n))$

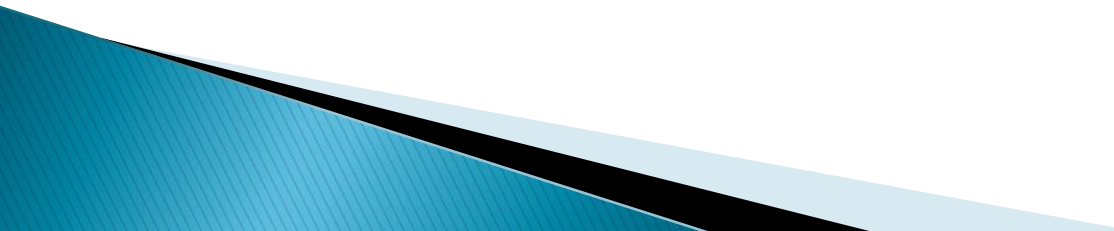
Exemplu: căutarea binară

- ▶ Complexitate liniară $O(n)$

Exemplu: minimul dintr-un vector

- ▶ Complexitate $O(n \log_2(n))$ liniară logaritmică

Exemplu: sortarea prin interclasare MergeSort



Timpul de executare a algoritmilor

- ▶ **Complexitate pătratică $O(n^2)$**

Exemplu: suma elementelor unei matrice $n \times n$,
sortarea prin metoda bulelor, prin selecție

- ▶ **Complexitate polinomială $O(n^k)$, $k \geq 3$**

Exemplu: înmulțirea a două matrice pătratice de dimensiune n

Timpul de executare a algoritmilor

- ▶ **Complexitate pătratică** $O(n^2)$

Exemplu: suma elementelor unei matrice $n \times n$,
sortarea prin metoda bulelor, prin selecție

- ▶ **Complexitate polinomială** $O(n^k)$, $k \geq 3$

Exemplu: înmulțirea a două matrice pătrate de dimensiune n

- ▶ **Complexitate exponențială** $O(k^n)$, $k \geq 2$

Exemplu: generarea submulțimilor unei mulțimi cu n elemente

- ▶ **Complexitate factorială** $O(n!)$

Exemplu: generarea permutărilor unui vector cu n elemente

Complexitate operații structuri de date in Python



Stiva

- ▶ Pentru o stivă operațiile de inserare și eliminare se fac la același capăt **LIFO**
- ▶ Putem implementa cu **list**
 - adăugare la final $O(1)$
 - eliminarea ultimului element $O(1)$

Stiva

```
ls_nr = [int(x) for x in input().split()]
stiva = []
for x in ls_nr:
    stiva.append(x)
for i in range(2):
    if len(stiva)>0:
        print(stiva.pop())#extrag ultimul element adaugat
```

Coada

- ▶ **FIFO** – inserare la un capăt, eliminare din celălalt
- ▶ Cu list – ineficient, una dintre operații este $O(n)$
- ▶ Variante:
 - clasa **deque** din modulul **collections**
 - clasa **Queue** din modulul **queue** (aceasta are operațiile sincronizate, de aceea sunt mai lente)
 - Operațiile de inserare și eliminare (la un capăt) pentru aceste tipuri au complexitate $O(1)$

Cooda

```
import collections

ls_nr = [int(x) for x in input().split()]

coada = collections.deque()

for x in ls_nr:
    coada.append(x)

for i in range(2):
    if len(coada) > 0:
        print(coada.popleft()) #extrage primul element adaugat
```

<https://docs.python.org/3.9/library/collections.html?highlight=deque#collections.deque>

Coadă

```
import queue

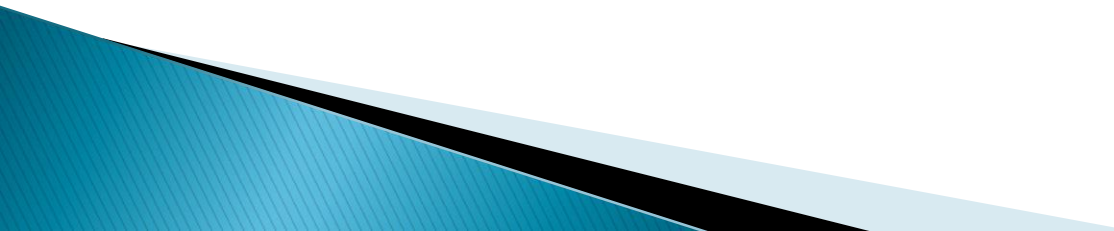
ls_nr = [int(x) for x in input().split()]

coada = queue.Queue()

for x in ls_nr:
    coada.put(x)

for i in range(2):
    if coada.qsize() > 0:
        print(coada.get())#extrage primul element adaugat
```

<https://docs.python.org/3/library/queue.html>



Coada de priorități

- ▶ eliminare – elementul minim în $O(\log(n))$
- ▶ Adăugare – tot $O(\log(n))$
- ▶ Variante:
 - modulul **heapq**
 - clasa **PriorityQueue** din modulul **queue** (sincronizata, mai lentă, se bazează tot pe **heapq**)

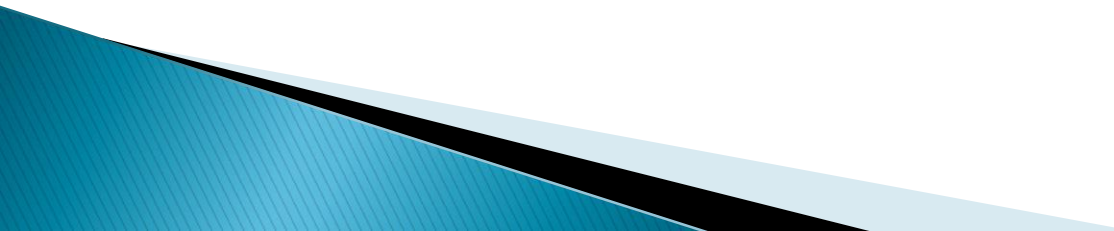
Coadă de priorităţi

```
import heapq

ls_nr = [int(x) for x in input().split()]
h = [] #un heap este un vector
for x in ls_nr:
    heapq.heappush(h, x)

for i in range(2):
    if len(h) > 0:
        print(heapq.heappop(h)) #extrage elementul minim
```

<https://docs.python.org/3/library/heapq.html>



Coadă de priorităţi

```
import queue

ls_nr = [int(x) for x in input().split()]

coada = queue.PriorityQueue()

for x in ls_nr:
    coada.put(x)

for i in range(2):
    if coada.qsize() > 0:
        print(coada.get())#extrage elementul minim
```

<https://docs.python.org/3/library/queue.html#>

